

HESAPLAMANIN SINIRLARI

Indirgeme · P · NP · NP-Complete · NP-Hard · P=NP?

Bazi problemler cok zor – cozum bulmak verimli gozukmez. Bu bolum hangi problemlerin ne kadar zor oldugunu siniflandirir. **Temel soru:** Cozumu bulmak mi daha zor, yoksa dogrulamak mi?

1. Bilinen Karmasiklik Sinirlari

Hesaplamanin Sinirlari – Genel Bakis

Problem / Algoritma	En Kotü Durum Karmasikligi
Arama algoritmaları (en kotü)	$O(\log n)$
Siralama algoritmaları (en kotü)	$O(n \log n)$
Dijkstra	$O((E + V) \log V)$
Floyd-Warshall	$O(V ^3)$
Bellman-Ford	$O(V \cdot E)$
Strassen'in hizli matris carpimi	$O(n^{2.81})$
Naif matris carpimi	$O(n^3)$

Bu algoritmalar polinom zamanda (P sinifinda) calisir. Asagida P sinifini asmaya baslayan problemleri gorecegiz.

2. Indirgeme (Reduction)

Indirgeme (Reduction)

Temel fikir: $A \leq_p B$ ifadesi, “ A problemi B problemine polinom zamanda indirgenebilir” demektir. Yani A ’yi cozmek icin B ’yi bir kez cagirip sonucu kullaniriz.

$A \leq_p B$ sembolu ne anlama gelir?

- A problemini polinom zamanda B problemine indirgeyebiliyoruz.
- Eger $B \in O(f(n))$ ise $A \in O(f(n))$ olur.
- Eger $A \in \Omega(f(n))$ ise $B \in \Omega(f(n))$ olur.

Sezgi: A , B ’den en fazla B kadar zordur. B kolaysa A de kolaydirlar. A zor ise B de en az A kadar zordur.

Ornek 1 – Kucukten buyuge siralama ile eslestirme:

Elimizde X ve Y gibi iki dizi olsun. X 'deki en kucuk degeri Y 'deki en kucuk degerle, X 'deki bir sonraki kucuk degeri Y 'deki bir sonraki en kucuk degerle eslestirerek iki dizideki ogelerin eslesmesi problemini ele alalim. Bu, siralama problemine indirgenabilir ($O(n \log n)$).

Ornek 2 – Carpma ile indirgeme:

n basamakli iki sayinin carpimi problemi ele alalim. Naif algoritma n^2 'lik adimla yapilir maliyeti $O(n^2)$ 'dir. Hızli carpma: **Karatsuba** (bol ve fethet).

$n \times n$ 'lik iki matrisin carpimi problemini de ele alalim. $A \cdot B$ carpimini hesaplamak icin $n \times n$ matrisleri $n/2 \times n/2$ alt matrislere bolerek:

$$A = \begin{pmatrix} A & 0 \\ A^T & 0 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & B \\ B & 0 \end{pmatrix} \Rightarrow 2n \times 2n \Rightarrow A \cdot B \text{ elde edilir}$$

Bu indirgeme sayesinde matris carpimini daha verimli matris carpim algoritmalarından yararlanarak gerçekleştirmek mümkündür.

Pratik – Indirgeme

(a) $A \leq_p B$ ve B polinom zamanda cozulebiliyorsa, A hakkında ne soyleyebiliriz? ____

(b) $A \leq_p B$ ve A NP-Hard ise, B hakkında ne cikarabilirsiniz? _____

3. Problem Turleri**Problem Turleri**

Karar Problemi (Decision Problem): Cevabi yalnızca “evet” ya da “hayır” olan problemlerdir.

Optimizasyon Problemi: Olasi cozumler kumesi uzerinde bir amac fonksiyonunun en iyi degerinin (minimum veya maksimum) arandigi problemlerdir.

Problem	Karar Versiyonu	Optimizasyon Versiyonu
Traveling Salesman (TSP)	6 sehir var, 300 km'den kısa bir turla gezilebilir mi?	Tum sehirleri kapsayan en kısa tur kac km'dir?
Knapsack	Bu canta kapasitesiyle toplam degeri 100 olan secim yapabilir miyim?	Bu canta kapasitesiyle alabileceğim maksimum deger nedir?
3-SAT	$(x_1 \vee x_2 \vee x_3) \wedge \dots$ formulu TRUE yapacak bir degisken ataması var mi?	OR'larla baglanmış 3'lu cumleciklerin kaci TRUE yapılabilir?

Knapsack Problemi detayi: Bir sirtcantamız var ve kapasitesi sinirli. Bunun disinda bir dizi esya var. Her birinin ağırligi ve degeri belli. Ancak tum esyalari alamayiz. Maksimum deger elde etmek istiyoruz; yalnız ağırlık sinirini asmadan. Buna gore bu esyalar arasında oyle bir secim yapabilir miyiz ki toplam ağırlık 15 kg'yi gecmesin, toplam deger en az 100 olsun?

3-SAT Problemi: $(x_1 \vee x_2 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_4) \wedge (x_2 \vee \overline{x_3} \vee x_5)$

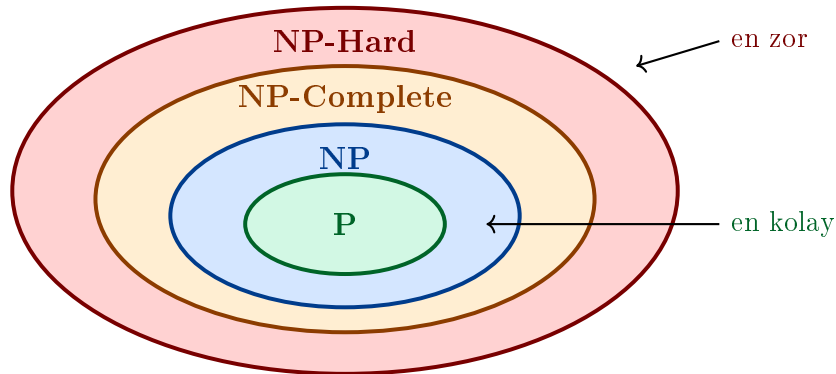
Tum bu OR'larla baglanmış 3'lu cumleciklerin hepsi ayni anda TRUE olur mu? Diger bir deyle bu formulü TRUE yapacak bir degisken ataması var mi?

Pratik – Problem Turleri

(a) TSP'nin karar versiyonu ile optimizasyon versiyonu arasındaki fark nedir? _____

(b) Knapsack'ın karar versiyonunu “evet/hayır” sorusu olarak yazın. _____

4. P, NP, NP-Complete, NP-Hard



P – Polinom Zaman Problemleri

P – Polinom Zaman Problemleri: Polinom zamanda cozulebilen problemlerdir. Yani bir deterministik Turing makinesiyle $O(n^k)$ adimda cozulebilir.

Ornekler: Siralama ($O(n \log n)$), kısa yol (Dijkstra), MST (Kruskal), binary search ($O(\log n)$), matris carpimi.

NP – Non-deterministik Polinom Zaman

NP – Non-deterministik Polinom Zaman Problemleri:

- Deterministik olmayan bir Turing makinesiyle polinom zamanda cozulebilen problemlerdir.
- Problemin cozumu veya cozumlerinin dogrulanmasi polinom zamanda Turing makinesiyle yapilabilen problemlerdir.
- Non-deterministik bir Turing makinesinin her adiminda mumkun olan tum hesaplama dallarini takip ettigi ve bu dallardan en az bir kabul durumuna ulasirsa makinenin bunu kabul ettigi hatirla.

Ornekler: SAT, Hamilton path, Knapsack, Vertex cover problemi.

Not: $P \subseteq NP$. Her P problemi NP 'dir cunku cozumu polinom zamanda bulursak, dogrulama da polinom zamanda yapilir.

NP-Complete Problemler

NP-Complete Problemler:

- Problem NP 'de olmalidir. Yani cozumu polinom zamanda dogrulanabilmelidir.
- NP 'deki her problem, bu probleme polinom zamanda indirgenebilmelidir. ($\forall L \in NP : L \leq_p$ bu problem)

Sezgi: NP-Complete problemler NP 'nin en zor problemleridir. Birini polinom zamanda cozersek hepsini cozmus oluruz.

Ornekler: TSP (Traveling Salesman Problem), Knapsack, 3-SAT, Graf renklendirme, bir grafın Hamilton dongusune sahip olup olmadigini belirleme.

Cook-Levin Teoremi: SAT, ilk NP-Complete problem olarak ispatlanmistir (1971). Butun NP-Complete problemler birbirine polinom zamanda indirgenebilir.

NP-Hard Problemler

NP-Hard Problemler:

NP 'deki her problemin polinom zamanda indirgenebilecegi problemlerdir. NP-Hard problemler NP 'de olmak zorunda degildir. Dolayisiyla cozumu polinom zamanda dogrulanamayabilir.

Fark: $NP\text{-Complete} = NP\text{-Hard} \cap NP$. Yani NP-Complete problemler hem NP-Hard hem de NP'dedir. NP-Hard ama NP-Complete olmayan problemler de vardir (ornegin: Halting Problem).

	NP'de mi?	NP-Hard mi?	Polinom dogrulama?
P	Evet	Hayir	Evet
NP	Evet	Hayir (genelde)	Evet
NP-Complete	Evet	Evet	Evet
NP-Hard	Belki hayir	Evet	Belki hayir

Pratik – P, NP, NP-Complete, NP-Hard

- (a) Binary search P'de midir? Neden? _____
- (b) Knapsack'in cozumunu nasil polinom zamanda dogrularsiniz? (NP'de oldugunu gosterin) _____
- (c) Bir problem hem NP-Complete hem NP-Hard olabilir mi? Aciklayin. _____
- (d) NP-Hard ama NP'de olmayan bir problem ornegi veriniz. _____

5. $P = NP$ mi?

Bilgisayar biliminin en buyuk acik sorularindan biri sudur:

Bir problemi cozmek, cozumu dogrulamaktan gercekten daha zor mudur? Sezgilerimize gore evet der. Ancak bu su anda kanitlanamamaktadır. 1971'den beri yanitlanamayan bu soru matematik ve bilgisayar biliminin seyrini degistirebilecegi gucedir.

Resmi ifade:

$$P \stackrel{?}{=} NP$$

Eger $P = NP$ olsaydi: Bir problemi cozmek ile cozumu dogrulamak arasindaki fark ortadan kalkardi. Yani SAT, TSP, Knapsack gibi NP-Complete problemler polinom zamanda cozulebilir hale gelirdi.

Eger $P \neq NP$ ise (cok daha buyuk olasilik): NP-Complete problemler icin polinom zamanli algoritma yoktur. Kriptografi, biyoinformatik ve optimizasyon problemleri “zor” kalmaya devam eder.

Scott Aaronson: <https://scottaaronson.blog/?p=122>

Yani $P = NP$ olsaydi, bir problemi cozmek ile cozumu dogrulamak arasindaki fark ortadan kalkardi.

Pratikte ne yapiyoruz? NP-Complete/NP-Hard problemler icin:

- Yaklasim algoritmaları (approximation algorithms): garanti edilmiş hata payı ile

hizli cozum.

- Heuristikler: genetik algoritmalar, simulated annealing.
- Ozel durum cozumleri: gercek hayat verisinin belirli ozellikleri kullanilarak verimlilestirme.

Pratik – $P = NP$?

(a) $P = NP$ olsaydi kriptografi nasil etkilenirdi? _____

(b) NP-Complete bir problemi cozmek icin polinom zamanli algoritma bulunursa bu ne anlama gelir? _____

Ozet:

- **P:** Polinom zamanda cozulebilir. (Hizli)
- **NP:** Polinom zamanda dogrulanabilir. ($P \subseteq NP$)
- **NP-Complete:** Hem NP'de, hem NP-Hard. En zor NP problemleri.
- **NP-Hard:** Her NP problemi buraya indirgenebilir; NP'de olmak zorunda degil.
- **Indirgeme:** $A \leq_p B$ – A 'yi cozmek icin B kullanilir.
- $P = NP?$: Bilgisayar biliminin en buyuk acik sorusu. 1971'den beri cevapsiz.